When using straight-line drawings, we can imagine vertices as ions, electrically charged particles: when there is an edge between two vertices, the particles attract each other (like if they had opposite charge), while pair of vertices not connected by edges repeal each other.

And then we can try to find an equilibrium point for the system, a disposition of the particles such that all the forces balance each other and the system can maintain a stable condition; sometimes instead of explicitly computing[18] the point of equilibrium, we can rather try to approximate it by simulating the evolution of the system using a heuristic.

Turns out, there is a whole class of graph embedding algorithms that adopt this principle: the so-called **force-directed graph drawing** algorithms.

The goal of these algorithms is to lay-down a graph’s vertices in the 2D space so that adjacent vertices are more or less at the same distance (and as such, all edges are of the same length in the plane) and, of course, there are as few edge intersections as possible; this is done by computing forces among the adjacent vertices (attractive forces), and among all non-adjacent vertices (repulsive), based on their relative positions, and then update the system (i.e. those positions) based on the forces computed and some parameters, trying to minimize the energy of the whole system.